FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 2: Values, variables, strings, and lists

Tim Van de Cruys



Wednesday 30 September 2026

Python as a calculator

- Computers are very good at number-crunching
- *Operators* (specially reserved symbols) represent mathematical computations:

Addition	`3 + 2`
Subtraction	`5 - 1`
Multiplication	`4 * 5`
Division	`20 / 2`
Exponentiation	`10 ** 2`
Modulus	`100 % 8`
Floor division	`4 // 3`

- The values the operator is applied to are called *operands*

Python as a calculator

One or more operators can be used to form an expression

```
In: 1 + 2 * 3
Out: 7
In: (1 + 2) * 3
Out: 9
```

Python as a calculator

One or more operators can be used to form an expression

```
In: 1 + 2 * 3
Out: 7
In: (1 + 2) * 3
Out: 9
```

Note:

- Python follows the precedence rules of mathematics (exponentiation before multiplication/division before addition/subtraction)
- Parentheses can be used to specify a different order

Numeric data types

- Values have a *type*
- Different types for numeric values:

  Integers (whole numbers): `1`, `2`, `3`, ..., `-1`, `-2`, `-3`

  Floating point numbers: `2.5`, `1.3333`, ...

Numeric data types

- Values have a *type*
- Different types for numeric values:

  Integers (whole numbers): `1, 2, 3, ..., -1, -2, -3`

  Floating point numbers: `2.5, 1.3333, ...`

```
In: 5 / 2
Out: 2.5
In: type(5 / 2)
Out: float
In: type(5 // 2)
Out: int
```

Variables

- A **variable** is a name that refers to a value
- A variable can be used to store the result of an expression so that it can be re-used
- A variable is created/updated using the equal sign ('=')

```
In: a = 5 / 2
In: a
Out: 2.5
```

- Name of the variable:
  - can be arbitrarily long
  - can contain both letters and numbers, but cannot start with a number
  - may also contain underscore character
  - is case-sensitive

Terminology

Constant e.g., `2`, `5`, `1.5`, ...

Expression A sequence of operands (constants) and operators

- `2 + 5`
- `1 + 2 + 3`
- etc.

Variable a container that holds a value, can be used in place of constants; e.g., `a`, `b`, `books`, ...

Using variables

We can use a variable in expressions instead of a constant:

```
In: a + 0.5
Out: 3.0
```

Using variables

We can use a variable in expressions instead of a constant:

```
In: a + 0.5
Out: 3.0
```

A program is executed line-by-line, and variables can be overwritten:

```
In: a = 2
In: a
Out: 2
In: a = 3
In: a
Out: 3
```

Updating variables

A common operation is to increase the value of a variable:

```
In: a = 2
In: a = a + 1
Out: 3
```

Updating variables

A common operation is to increase the value of a variable:

```
In: a = 2
In: a = a + 1
Out: 3
```

There is a shorthand for this operation:

```
In: a = 2
In: a += 1
Out: 3
```

Also `-=`, `*=`, etc.

An example: Reading Proust

How many months does it take me to read all seven volumes of Marcel Proust's novel *À la recherche du temps perdu* (In Search of Lost Time)?

- Number of words in *La Recherche*: 1,267,069
- Time spent reading per day: 1 hour
- Number of words per minute: 238

How to calculate the total reading time in months?

Text

Values can also be text instead of numbers:

```
author = 'Marcel Proust'
movie = "Breakfast at Tiffany's"
lyrics = """When I find myself in times of trouble
            Mother Mary comes to me
            Speaking words of wisdom, let it be"""
```

- A **string** (`str`, short for string of characters) is the type used for text
- Single quotes, double quotes, or three quotation marks for different usages

Types

- We have seen numbers (`int`, `float`) and text (`str`)
- These are examples of types
- A **type** is a class of values recognized by the language
- In Python, a value always has a type (a variable can be any type)
- The type determines what operations are valid

Different behavior for types

```
In: 3 + 3
Out: 6
```

These are numbers, so we can do arithmetic

```
In: '3' + '3'
Out: '33'
```

These are strings, so they are treated as text

Invalid operations for types

```
In: 3 + '3'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Converting numbers to text and back

```
In: number = 42
In: text = str(number)
In: text
Out: '42'

In: int(text)
Out: 42
```

- `str(number)` and `int(text)` are examples of using a function
  - A **function** is a piece of code that can be invoked by name
  - It may take input (arguments)
  - It may produce output (return value)
  - The notation `f(x)` means that the function `f` is *applied* to the value `x`

Summary

- Values and types:

  Numbers: `int`, `float`

  Text: `str`

- Expressions: `1 + 2 * 3 - (4 + 5)`
- Variables: `a = 2`
- Functions: `str(2)`, `int('2')`

Comments

- Large piece of code can be difficult to read
- It helps to explain what you're doing
- Explanations are called *comments*
- Comments start with hash (#)
- Everything behind the # is ignored by the Python interpreter

```
c = (f - 32) * (5/9)
```

Comments

- Large piece of code can be difficult to read
- It helps to explain what you're doing
- Explanations are called *comments*
- Comments start with hash (#)
- Everything behind the # is ignored by the Python interpreter

```
temp_c = (temp_f - 32) * (5/9) #convert fahrenheit to celsius
```

Example: computing readability

- Automated readability index
- A formula to estimate the difficulty of a text

$$0.50\left(\frac{\text{total words}}{\text{total sentences}}\right) + 4.71\left(\frac{\text{total characters}}{\text{total words}}\right) - 21.43$$

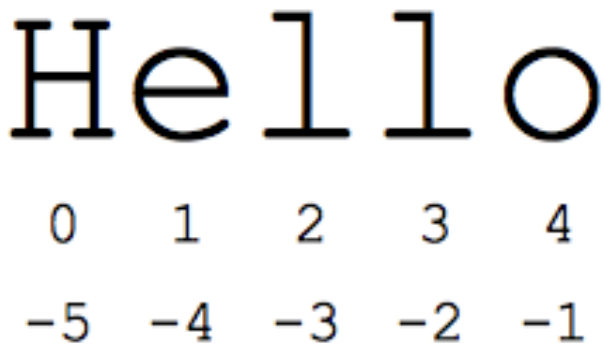
- Result: a grade level, e.g.
  - 1 = kindergarten, age 5
  - 14 = college student, age 18

Smith & Senter (1968). Automated readability index.
`https://apps.dtic.mil/dtic/tr/fulltext/u2/667273.pdf`

Indexing: extracting one element

```
In: name = 'Marcel'
In: name[0]
Out: 'M'
In: name[-1]
Out: 'l'
```

- `a[n]` gets the n'th character of a
- Counting starts at zero
- negative indices start from the end
  - -1 for last character
  - -2 second to last
  - etc.

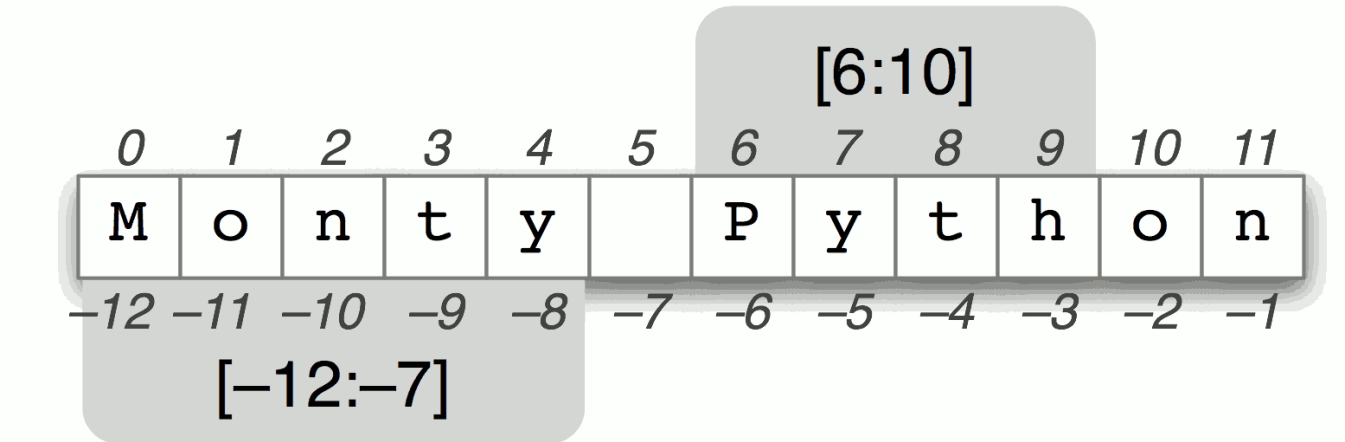


Slicing: extracting a range of elements

```
In: name = 'Marcel'
In: name[1:3]
Out: 'ar'
In: name[:2]
Out: 'Ma'
In: name[1:]
Out: 'arcel'
```

- `a[n:m]` extracts characters n to m
- Counting starts at zero
- Result is up to but not including character m (half-open interval)
- if n or m is left out, beginning or end is used

Indexing and slicing



Length

Get the number of characters in a string:

```
In: name = 'Marcel'
In: len(name)
Out: 6
```

Functions

We have seen a number of **functions**:

- `int()`
- `print()`
- `len()`

- Reminder:
  - A **function** is a piece of code that can be invoked by name
  - It may take input (arguments): `len(name)`
  - It may produce a result: `len(name)` or an effect: `print('hello', name)`
  - The notation `f(x)` means that the function `f` is *applied* to the value `x`

- The functions above are **builtin functions**: they are defined by default by the Python interpreter
- You can also define your own functions (more on that later)

Format strings

```
In: x = 1
In: y = 2
In: z = x + y
```

How to combine text and variables in a message?

```
In: print('The sum of ' + str(x) + ' and ' + str(y) + ' is ' + str(z) +
The sum of 1 and 2 is 3.
In: print('The sum of', x, 'and', y, 'is', z, '.')
The sum of 1 and 2 is 3 .
```

Format strings

```
In: x = 1
In: y = 2
In: z = x + y
```

How to combine text and variables in a message?

```
In: print('The sum of ' + str(x) + ' and ' + str(y) + ' is ' + str(z) +
The sum of 1 and 2 is 3.
In: print('The sum of', x, 'and', y, 'is', z, '.')
The sum of 1 and 2 is 3 .
```

Or using a formatted string:

```
In: print(f'The sum of {x} and {y} is {z}.')
The sum of 1 and 2 is 3.
```

Sequences

- A string is a kind of sequence consisting of characters
- A sequence contains elements in a particular order
- Basic operations on sequences:

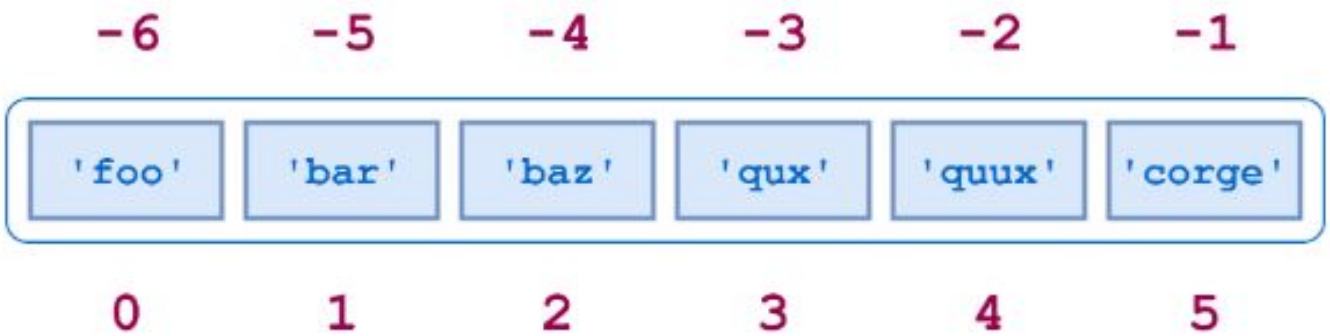
length: `len(seq)`
indexing: `seq[index]`
slicing: `seq[start:end]`
concatenation: `seq1 + seq2`
repetition: `seq * num`

Lists

- Lists are sequences that may contain any kind of value as items

```
In: numbers = [0, 1, 2]
In: names = ['John', 'Mary']
```

List indexing



Changing items

```
In: names = ['John', 'Mary']
In: names[1] = 'Alice'
In: names
Out: ['John', 'Alice']
```

- But: characters in a string cannot be changed
- Necessary to create a new string

```
In: name = 'Maria'
In: name[0] = 'D'
TypeError: 'str' object does not support item assignment
```

- Why? Lists are mutable, strings are immutable
- **immutable** objects do not allow modification after creation

Immutable vs. mutable

Immutable

- Sometimes we want to ensure objects cannot be changed
- Immutable objects are quicker to access

Mutable

- Size and content of objects can be changed
- More efficient if we need to change/update the contents

Objects and their methods

- In Python, every value is an object
- An **object** encapsulates data and code for dealing with that data
- A function that belongs to and operates on an object is called a **method**

Regular function: `function(argument)`

Method of object: `obj.function(argument)`

Adding items

- We can add an item to the end of an existing list:

```
In: names = ['John', 'Mary']
In: names.append('Alice')
In: names
Out: ['John', 'Mary', 'Alice']
```

- We can combine two lists into a new list:

```
In: friends = ['John', 'Mary']
In: enemies = ['Alice', 'Bob']
In: friends + enemies
Out: ['John', 'Mary', 'Alice', 'Bob']
```

- Note: `append` mutates the list, `+` only creates a new list

Removing items

- `remove(elem)` looks for the first occurrence of `elem` and removes it from the list

```
In: names = ['John', 'Mary']
In: names.remove('John')
In: names
Out: ['Mary']
```

- `.pop(value)` removes the item at position `value` and returns it
- `.pop()` (without argument) removes the last items and returns it

```
In: names = ['John', 'Mary', 'Alice', 'Bob', 'Max']
In: names.pop(3)  # removes 4th item
Out: 'Bob'
In: names.pop()   # removes last item
Out: 'Max'
In: names
Out: ['John', 'Mary', 'Alice']
```

Removing items (2)

- We can also use slicing to keep a part of a list

```
In: names = ['John', 'Mary', 'Alice', 'Bob']
In: friends = names[:2]
In: friends
Out: ['John', 'Mary']
```

Sorting

```
In: letters = ['b', 'd', 'a', 'c']
In: letters
Out: ['b', 'd', 'a', 'c']
In: sorted(letters)
Out: ['a', 'b', 'c', 'd']
In: letters
Out: ['b', 'd', 'a', 'c']
In: letters.sort()
In: letters
Out: ['a', 'b', 'c', 'd']
```

- The function `sorted()` leaves the original list unchanged and returns the sorted result
- The method `.sort()` modifies the list itself, and does not return anything

Nested lists: lists inside of lists

```
In: ages = [['John', 34], ['Mary', 19], ['Max', 62]]
```

- How to access elements?

```
In: name_and_age = ages[0]
In: name_and_age
Out: ['John', 34]
In: name_and_age[1]
Out: 34
```

Nested lists: lists inside of lists

```
In: ages = [['John', 34], ['Mary', 19], ['Max', 62]]
```

- How to access elements?

```
In: name_and_age = ages[0]
In: name_and_age
Out: ['John', 34]
In: name_and_age[1]
Out: 34
```

```
In: ages[0][1]
Out: 34
```

Summary: strings vs lists

Both strings and lists are sequences:

- Items in a fixed order
- Has a length
- Can extract items or slices

Two important differences:

1. lists are more general than strings
   - Strings always consist of characters
   - Lists can consist of any data types / objects
2. Lists are mutable and strings are immutable

Practical session / reading

- First practical session
  - Notebook is self-explanatory: **self-study** with exercises
  - Core exercises first; challenge exercises at the end if you want more
  - Help will be available during practical session slot (today, 4pm-6pm, VHI 02.25)
  - Questions can be asked during practical session or online
- The code-along notebook of this lecture is on Toledo: re-run the examples at home
- Read chapters 3 & 4 of Python for Everybody